ИнфТех
ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ

# Программные компоненты КомпоГраф

## Руководство разработчика

Релиз 1.0
страниц 58

Москва 2023

**Содержание:**

## Оглавление


Введение ..... 5

Компоненты ..... 5

Кнопки ..... 5

Кнопки для окон и сообщений ..... 5

Основные кнопки ..... 5

Основные кнопки с иконками ..... 6

Кнопки с иконками ..... 6

Флажок - kg-checkbox ..... 7

Описание ..... 8

Динамическое добавление: Пример ..... 8

Событие ..... 8

Кнопка включатель - kg-checkbox-button ..... 8

Внешний вид ..... 8

Интерфейс / Свойства ..... 9

Пример использования с преобразованием типа ..... 9

Описание ..... 9

Динамическое добавление: Пример ..... 9

Событие ..... 10

Выключатель - kg-switch ..... 10

Внешний вид ..... 10

Интерфейс / Свойства ..... 10

Динамическое добавление: Пример ..... 11

Событие ..... 11

Переключатель - kg-radio ..... 12

Внешний вид ..... 12

Интерфейс / Свойства ..... 12

Описание ..... 13

Скрипт ..... 13

Кнопка переключатель - kg-radio-button ..... 14

Внешний вид ..... 14

Интерфейс / Свойства ..... 14

Описание ..... 15

Событие ..... 15

Скрипт....15
Плавающая кнопка - kg-action-button....16
Выпадающий список с кнопками - kg-dropdown....19
Панель кнопок - kg-toolbar....22
Внешний вид....22
Поле ввода....26
Поле input....26
Поле textarea....26
Поле со списком - kg-select....27
Внешний вид....27
Разметка....27
База данных....28
Скрипт....28
Интерфейс / Свойства....29
Описание....30
Свойства....30
Методы....30
События....31
Поле ввода число - kg-number....31
Внешний вид....31
Интерфейс / Свойства....32
Описание....33
Событие....34
Скрипт....34
Поле поиска - kg-search....34
Внешний вид....34
Простое поле....34
Поле с кнопками группировки....34
Поле с кнопками группировки и сортировки....34
Интерфейс / Свойства....35
Описание....35
События....35
Скрипт....35
Выбор даты - kg-datepicker....36
Внешний вид....36
Интерфейс / Свойства....36

Описание....37
Динамическое добавление: Пример....38
События....38
Скрипт....38
Диапазон - kg-range....39
Внешний вид....39
Интерфейс / Свойства....39
Пример использования с преобразованием типа....40
Описание....40
Событие....41
Скрипт....41
Прогресс бар - kg-progress....42
Внешний вид....42
Интерфейс / Свойства....42
Описание....43
Работа с прогресс баром....43
Скрипт....43
Диаграммы....43
Начало работы....43
Создание с добавлением элемента через html....43
Создание с добавлением элемента через скрипт....44
Общее....45
Отступы графика....46
Работа с данными....47
Конфигурация заголовка....48
Конфигурация легенды....49
Конфигурация Осей....50

## Введение

Для использования компонентов требуется подключить ниже указанные библиотеки, css и js/ts.

Соблюдение порядка подключения библиотек обязательно!

Весь скрип написан на *.ts с наличием интерфейсов для компонентов

```
<head>
 <link rel="stylesheet" href = "~/css/kg.css" /> <!-- Таблица стилей -->
 <!-- Ваши стили -->
 <script src="~/js/widget/kg.library.js"> </script> <!-- Библиотека классов и
методов -->
 <script src="~/js/widget/kg.modals.js"> </script> <!-- Модального окно -->
 <script src="~/js/widget/kg.elements.js"> </script> <!-- Компоненты -->
<!-- Ваши скрипты -->
</head>
```

Пример использования цветов в css:

```
.blok{
    background-color: var(--color-primary);
    border: 1px solid var(--color-primart-4);
    color: var(--color-secondary);
}
```

## Компоненты

### Кнопки

#### Кнопки для окон и сообщений

Выбрать Выбрать Выбрать Выбрать

```
<div class="kg-tools">
   <button class="kg-button">Выбрать</button>
   <button class="kg-button kg-primary">Выбрать</button>
   <button class="kg-button kg-red">Выбрать</button>
   <button class="kg-button kg-green">Выбрать</button>
</div>
```

#### Основные кнопки

Выбрать Выбрать Выбрать Выбрать

```
<div class="kg-tools">
   <button class="kg-button-simple">Выбрать</button>
   <button class="kg-button-simple kg-primary">Выбрать</button>
   <button class="kg-button-simple kg-red">Выбрать</button>
   <button class="kg-button-simple kg-green">Выбрать</button>
</div>
```

## Основные кнопки с иконками

Выбрать Выбрать Выбрать Выбрать

```
<div class="buttons-blok">
    <button class="kg-button-simple">
      <svg class="kg-button-icon">
          <use href="/images/svg/dpicons.svg#speedtest"></use>
      </svg>
      Выбрать
    </button>
    <button class="kg-button-simple kg-primary">
      <svg class="kg-button-icon">
          <use href="/images/svg/dpicons.svg#slow-load"></use>
      </svg>
      Выбрать
    </button>
    <button class="kg-button-simple kg-red">
      <svg class="kg-button-icon">
          <use href="/images/svg/dpicons.svg#fast-download"></use>
      </svg>
      Выбрать
    </button>
    <button class="kg-button-simple kg-green">
      <svg class="kg-button-icon">
          <use href="/images/svg/dpicons.svg#donut-chart"></use>
      </svg>
      Выбрать
    </button>
</div>
```

## Кнопки с иконками

```
<div class="buttons-blok">
    <button class="kg-icon-button">
      <svg class="kg-icon-button-svg">
          <use href="/images/svg/dpicons.svg#add"></use>
      </svg>
    </button>
    <button class="kg-icon-button kg-primary">
      <svg class="kg-icon-button-svg">
          <use href="/images/svg/dpicons.svg#ok"></use>
    </button>
    <button class="kg-icon-button kg-red">
      <svg class="kg-icon-button-svg">
          <use href="/images/svg/dpicons.svg#double-tick"></use>
      </svg>
    </button>
```

```
    <button class="kg-icon-button kg-green">
      <svg class="kg-icon-button-svg">
          <use href="/images/svg/dpicons.svg#close"></use>
      </svg>
    </button>
</div>
```

### Флажок - kg-checkbox

Без текста

С текстом

Текст включателя

```
<div class="kg-form-row">
    <h3 class="kg-form-row-label">Без текста</h3>
    <kg-checkbox value="true"></kg-checkbox>
</div>
<div class="kg-form-row">
    <h3 class="kg-form-row-label">С текстом</h3>
    <kg-checkbox value="true" text="Текст включателя"></kg-checkbox>
</div>
```

### Интерфейс / Свойства

```
/** Включатель */
interface DPElementCheckBox extends HTMLElement {
  /**Текст отображаемый справа */
  text: string; //get/set
  /**Значение true/false: по умолчанию - false */
  value: boolean; //get/set
  /**Заблокировать элемент. true - заблокировать, false - снять блокировку: по
умолчанию - false  */
  disabled: boolean; //get/set
  /** Подсказка элемента */
  title: string; //get/set
}
```

### Пример использования с преобразованием типа

```
let element = document.querySelector("kg-checkbox") as DPElementCheckBox;

element.value = false;
let value = element.value;

element.disabled = true;
let disabled = element.disabled;

element.text = "Новый текст";
let text = element.text;

element.title = "Новая подсказка";
```

```
let title = element.title;
```

### Описание

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **disabled** | boolean Заблокировать элемент | Да | Да | Да | false |
| **text** | string Текст отображаемый справа | Да | Да | Да | |
| **title** | string Подсказка элемента | Да | Да | Да | |
| **value** | boolean Значение true/false | Да | Да | Да | false |

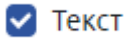
### Динамическое добавление: Пример

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let element = document.createElement("kg-checkbox") as DPElementCheckBox;
element.title = "Текс выключателя";
element.value = true;
```

### Событие

*Разметка*




```
<kg-checkbox value="true" id="textCheckbox" text="Текст"></kg-checkbox>
```

*Скрипт*

```
let checkbox = document.getElementById("textCheckbox") as DPElementCheckBox;
if (checkbox) {
    checkbox.addEventListener("changed", (e) => {
        let target = e.target as DPElementCheckBox;
         if (target) {
             console.log(target.value);
         }

     })
}
```

## Кнопка включатель - kg-checkbox-button

### Внешний вид

*Кнопка включатель с 1-ой иконкой*

*Кнопка включатель с 2-мя иконками*

```
<div class="kg-form-row">
     <kg-checkbox-button value="true" icon="remove" title="Удалить"></kg-checkbox-
button>
</div>
<div class="kg-form-row">
```

```
    <kg-checkbox-button value="true" icon="remove" iconReplace="add"
title="Удалить"></kg-checkbox-button>
</div>
```

### Интерфейс / Свойства

```
interface DPElementCheckboxButton extends HTMLElement {
   /** текст, отображаемый при наведении на компонент */
   title: string;
   /** значение блокировки компонента(true - заблокированы, false - активный). По
умолчанию - false */
   disabled: boolean;
   /** id иконки компонента из спрайта. */
   icon: string;
   /** id подменной иконки компонента из спрайта. */
   iconReplace: string;
   /** значение "checked" внутреннего input type "checkbox" элемента. По умолчанию -
false. */
   value: boolean;
   /** значение действия, которое должно выполниться, при нажатии на компонент. */
   action(): string;
}
```

### Пример использования с преобразованием типа

```
const element = document.querySelector("kg-checkbox-button") as
DPElementCheckboxButton;

element.title = "Текст";
 const title = element.title;

element.disabled = false;
const disabled = element.disabled;

element.icon = "id_иконки_в_спрайте";
const icon = element.icon;

element.value = true;
const value = element.value;
```

### Описание

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **action** | string Действие элемента | Да | Да | Да | |
| **disabled** | boolean Заблокировать элемент | Да | Да | Да | false |
| **icon** | string Иконка элемента | Да | Да | Да | |
| **iconReplace** | string Подменная иконка элемента | Да | Да | Да | |
| **title** | string Подсказка элемента | Да | Да | Да | |
| **value** | boolean Значение true/false | Да | Да | Да | false |

### Динамическое добавление: Пример

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
 let element = document.createElement("kg-checkbox-button") as
DPElementCheckboxButton;
```

```
element.title = "Текст выключателя";
element.icon = "id_иконки_в_спрайте";
element.value = true;
```

Событие

*Разметка*

```
<kg-checkbox-button id="checkboxButton" value="true" icon="remove" icon="add"
title="Удалить"></kg-checkbox-button>
```

*Скрипт*

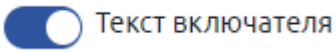
```
let checkboxButton = document.getElementById("checkboxButton") as
DPElementCheckboxButton;
if (checkboxButton) {
    checkboxButton.addEventListener("changed", (e) => {
        let target = e.target as DPElementCheckboxButton;
         if (target) {
             console.log(target.value);
         }
     });
}
```

Выключатель - kg-switch

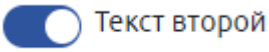
Внешний вид

*Без текста*

*С текстом*




*С переключением текста*




```
<div class="kg-form-row">
    <h3 class="kg-form-row-label">Без текста</h3>
    <kg-switch value="true"></kg-switch>
</div>
<div class="kg-form-row">
    <h3 class="kg-form-row-label">С текстом</h3>
    <kg-switch value="true" text="Текст включателя"></kg-switch>
</div>
<div class="kg-form-row">
    <h3 class="kg-form-row-label">С переключением текста</h3>
    <kg-switch value="true" text="Текст первый" second="Текст второй"></kg-switch>
</div>
```

Интерфейс / Свойства

```
interface DPElementSwitch extends HTMLElement {
  /**Текст отображаемый справа */
  text: string; //get/set
```

```
  /**Второй текст отображаемый справа */
  second: string; //get/set
  /**Значение true/false */
  value: boolean; //get/set
  /**Заблокировать элемент. true - заблокировать, false - снять блокировку */
  disabled: boolean; //get/set
  /** Подсказка элемента */
  title: string; //get/set
}
```

*Пример использования с преобразованием типа*

```
let switchElement = document.querySelector("kg-switch") as DPElementSwitch;

let title = switchElement.title;
switchElement.title = "Подсказка";

let text = switchElement.text;
switchElement.text = "Текст";

let second = switchElement.second;
switchElement.second = "Второй текст";

let disabled = switchElement.disabled;
switchElement.disabled = false;

let value = switchElement.value;switchElement.value = false;
```

*Описание*

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **disabled** | Заблокировать элемент | Да | Да | Да | false |
| **second** | Второй текст отображаемый справа | Да | Да | Да | |
| **text** | Текст отображаемы справа | Да | Да | Да | |
| **title** | Подсказка элемента | Да | Да | Да | |
| **value** | Значение | Да | Да | Да | false |

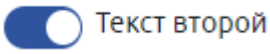
Динамическое добавление: Пример

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let element = document.createElement("kg-switch") as DPElementSwitch;
element.title = "Подсказка";
element.disabled = true
```

Событие

*Разметка*




```
 <kg-switch id="testId" value="true" text="Текст первый" second="Текст второй"></kg-switch>
```

*Скрипт*

```
let checkbox = document.getElementById("testId") as DPElementSwitch;
if (checkbox) {
```

```
    checkbox.addEventListener("changed", (e) => {
        let target = e.target as DPElementSwitch;
         if (target) {
             console.log(target.value);
         }
     })
}
```

## Переключатель - kg-radio

### Внешний вид

*Без текста*

*С текстом*

Текст 1
Текст 2

```
<div class="kg-form-row">
  <h3 class="kg-form-row-label">Без текста</h3>
  <kg-radio name="firstview" checked="true"></kg-radio> <!--переключатель Дataплат--
>
   <kg-radio name="firstview" ></kg-radio> <!--переключатель Дataплат-->
</div>
<div class="kg-form-row">
  <h3 class="kg-form-row-label">С текстом</h3>
  <kg-radio name="views" checked="true" text="Текст 1"></kg-radio> <!--переключатель
Дataплат-->
  <kg-radio name="views" text="Текст 2"></kg-radio> <!--переключатель Дataплат-->
</div>
```

### Интерфейс / Свойства

```
/** Переключатель */
interface DPElementRadio extends HTMLElement {
  /**Текст отображаемый справа */
  text: string; //get/set
  /**Имя значения */
  name: string; //get/set
  /**Значение */
  value: string; //get/set
  /**Состояние true/false */
  checked: boolean; //get/set
  /**Заблокировать элемент. true - заблокировать, false - снять блокировку: по
умолчанию - false  */
  disabled: boolean; //get/set
  /** Подсказка элемента */
  title: string; //get/set
}
```

*Пример использования с преобразованием типа*

```
let element = document.querySelector("kg-radio") as DPElementRadio;
```

```
element.value = "test";
let value = element.value;

element.disabled = true;
let disabled = element.disabled;

element.text = "Новый текст";
let text = element.text;

element.checked = true;
let checked = element.checked;

element.name = "Новый текст";
let name = element.name;

element.title = "Новая подсказка";
let title = element.title;
```

Описание

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **checked** | Состояние true/false | Да | Да | Да | false |
| **disabled** | Заблокировать элемент | Да | Да | Да | false |
| **name** | Имя значения | Да | Да | Да | |
| **text** | Текст отображаемы справа | Да | Да | Да | |
| **title** | Подсказка элемента | Да | Да | Да | |
| **value** | Значение | Да | Да | Да | |

*Динамическое добавление: Пример*

Для использования в TypeScript требуется преобразование типа ключевым словом as ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let element = document.createElement("kg-radio") as DPElementRadio;
element.text = "Текс выключателя";
element.checked = true;
```

*Событие*

Разметка

Текст 1
Текст 2

```
<div class="kg-form-row" id="radioElements">
   <kg-radio value="firstname" name="name" text="Текст 1"></kg-radio>
    <kg-radio value="secondname" name="name" text="Текст 2"></kg-radio>
</div>
```

Скрипт

```
let blok = document.getElementById("radioElements");
if (blok) {
    let radios = blok.querySelectorAll("kg-radio");
    if (radios) {
        for (let item of radios) {
            let radio = item as DPElementRadio;
            radio.addEventListener("changed", (e) => {
               let target = e.target as DPElementRadio;
```

```
                if(target) {
                console.log(target.value)
                }
            });
        }
    }
}
```

## Кнопка переключатель - kg-radio-button

### Внешний вид

*Кнопка переключатель*

```
<kg-radio-button id="radioButton" icon="remove" title="Удалить" name="radio" > </kg-radio-button>
<kg-radio-button icon="add" title="Добавить" name="radio"></kg-radio-button>
```

### Интерфейс / Свойства

```
interface DPElementRadioButton extends HTMLElement {
   /** текст, отображаемый при наведении на компонент */
   title: string;
   /** значение блокировки компонента(true - заблокированы, false - активный). По умолчанию - false */
   disabled: boolean;
   /** id иконки компонента из спрайта. */
   icon: string;
   /** Значение атрибута name внутреннего input type="radio" элемента. */
   name: string;
   /** значение "checked" внутреннего input type "checkbox" элемента. По умолчанию - false. */
   value: boolean;
   /** значение действия, которое должно выполниться, при нажатии на компонент. */
   action(): string;
}
```

*Пример использования с преобразованием типа*

```
const element = document.querySelector("kg-radio-button") as DPElementRadioButton;

element.title = "Текст";
 const title = element.title;

element.disabled = false;
const disabled = element.disabled;

element.name = "Текст";
const _name = element.name;

element.icon = "id_иконки_в_спрайте";
const icon = element.icon;
```

```
element.value = true;
const value = element.value;

element.action = "Текст";
const action = element.action;
```

Описание

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **action** | string Действие элемента | Да | Да | Да | |
| **disabled** | boolean Заблокировать элемент | Да | Да | Да | false |
| **icon** | string Иконка элемента | Да | Да | Да | |
| **name** | string Значение атрибута name внутреннего input type="radio" элемента | Да | Да | Да | |
| **title** | string Подсказка элемента | Да | Да | Да | |
| **value** | boolean Значение true/false | Да | Да | Да | false |

*Динамическое добавление: Пример*

Для использования в TypeScript требуется преобразование типа ключевым словом as ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
 let element = document.createElement("kg-radio-button") as DPElementRadioButton;
 element.title = "Текст переключателя
 element.icon = "id_иконки_в_спрайте";
 element.action = "Текст";
 element.name = "Текст";
```

Событие

Разметка

```
<kg-radio-button id="radioButton" icon="remove" title="Удалить" name="radio"></kg-radio-button>
<kg-radio-button icon="add" title="Добавить" name="radio"></kg-radio-button>
```

Скрипт

```
let radioButtonRemove = document.getElementById("radioButtonRemove") as DPElementRadioButton;
let radioButtonAdd = document.getElementById("radioButtonAdd") as DPElementRadioButton;
if (radioButtonRemove && radioButtonAdd) {
    radioButtonRemove.addEventListener("changed", (e) => {
        let target = e.target as DPElementRadioButton;
         if (target) {
             console.log(target.value);
         }
    radioButtonAdd.addEventListener(radioButton.action, (e) => {
        let target = e.target as DPElementRadioButton;
         if (target) {
```

```
            console.log(target.value);
        }
    })
}
```

## Плавающая кнопка - kg-action-button

### *Внешний вид*

**Важно**: по умолчанию компоненту заданы стили position: absoute, bottom: 10px, rigth: 10px. Родительскому контейнеру должен быть задан стиль position:relative

Плавающая кнопка с одной иконкой, без дочерних кнопок

Плавающая кнопка для добавления с дочерними кнопками

Плавающая кнопка с двумя иконками и дочерними кнопками

```
<div class="kg-form-row">
     <kg-action-button icon="remove" action="remove"></kg-action-button>
<div class="kg-form-row">
    <kg-action-button add="true">
        <ul>
            <li>
                <p>Добавить 1</p>
                <button data-action="add-1">
                    <svg class="kg-icon-button-svg">
                        <use href="/images/svg/dpicons.svg#add"></use>
                    </svg>
                </button>
            </li>
            <li>
                <p>Добавить 1</p>
                <button data-action="add-2">
                    <svg class="kg-icon-button-svg">
                        <use href="/images/svg/dpicons.svg#add"></use>
                    </svg>
                </button>
            </li>
        </ul>
    </kg-action-button>
</div>
<div class="kg-form-row">
    <kg-action-button icon="remove" iconReplace="close">
        <ul>
            <li>
                <p>Удалить 1</p>
```

```
                <button data-action="remove-1">
                    <svg class="kg-icon-button-svg">
                        <use href="/images/svg/dpicons.svg#remove"></use>
                    </svg>
                </button>
            </li>
            <li>
                <p>Удалить 1</p>
                <button data-action="remove-2">
                    <svg class="kg-icon-button-svg">
                        <use href="/images/svg/dpicons.svg#remove"></use>
                    </svg>
                </button>
            </li>
        </ul>
    </kg-action-button>
</div>
```

*Интерфейс / Свойства / Методы*

```
interface DPElementActionButton extends HTMLElement {
   /** значение id иконки в спрайте */
   icon: string;
   /** значение id подменной иконки в спрайте */
   iconReplace: string;
   /** значение режима кнопки - true/false(true - кнопка для добавления, id иконки в
спрайте - add, false - кнопка для чего-то другого). По умолчанию - false */
   add: boolean;
   /** значение блокировки компонента(true - заблокированы, false - активный). По
умолчанию - false */
   disabled: boolean;
   /** id иконки компонента из спрайта. */
   icon: string;
   /** значение "checked" внутреннего input type "checkbox" элемента. По умолчанию -
false. */
   value: boolean;
   /** значение действия, которое должно выполниться, при нажатии на
компонент/главной кнопке. */
   action(): string;
   /** получить значение id атрибута нажатой кнопки. */
   selectedId(): string;
   /** получить значение data-action(action в случае главной кнопки) атрибута
нажатой кнопки. */
   selectedAction(): string;
   /** блокировка/включение дочерней кнопки компонента
   * @param action - значение атрибута data-action кнопки, которую нужно
заблокировать/включить
   * @param status - true - для блокировки, false - для включения
   */
   Disable(action: string, status: boolean): Function;
}
```

### Пример использования с преобразованием типа

```
const element = document.querySelector("kg-action-button") as DPElementActionButton;
element.disabled = "true";
const disabled = element.disabled;
```

### *Свойства*

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **action** | string Действие элемента(главной кнопки) | Да | Да | Да | |
| **add** | boolean Значение режима ккомпонента | Да | Нет | Да | false |
| **disabled** | boolean Заблокировать элемент(главную кнопку) | Да | Да | Да | false |
| **icon** | string Значение id иконки в спрайте | Да | Нет | Да | |
| **iconReplace** | string Значение id подменной иконки в спрайте | Да | Нет | Да | |
| **selectedAction** | string значение data-action(action в случае главной кнопки) атрибута нажатой кнопки | Да | Нет | Нет | |
| **selectedId** | string значение id атрибута нажатой кнопки | Да | Нет | Нет | |

### *Методы*

| Название | Описание | Параметры | Возврат | Async |
|---|---|---|---|---|
| **Disable** | Блокировка/включение дочерней кнопки компонента | **action** - Значение атрибута data-action кнопки, которую нужно заблокировать/включить **status** - true - для блокировки, false для включения | Нет | Нет |

### *Динамическое добавление: Пример*

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let element = document.createElement("kg-action-button") as DPElementActionButton;
element.icon = "id_иконки_в_спрайте";
element.iconReplace = "id_иконки_в_спрайте";
```

### *События*

У компонента есть основное событие **"changed"**, которое вызывается при клике на дочерние кнопки в случае, если дочерних кнопок нет, событие вызывается при клике на основную кнопку.

Также при клике на дочернюю кнопку и наличии у нее дата-атрибута data-action вызывается событие, которое формируется из приставки "dp" + значения дата-атрибута data-action. Например, "dp" + "remove1" = **"dpremove1"**. В случае, если дочерних кнопок нет, событие формируется из строк "dp" + значение атрибута action компонента.

### Разметка

```
<kg-action-button add="true">
    <ul>
        <li>
```

```
            <p>Добавить 1</p>
            <button data-action="add-1">
                <svg class="kg-icon-button-svg">
                     <use href="/images/svg/dpicons.svg#add"></use>
                </svg>
            </button>
        </li>
        <li>
            <p>Добавить 1</p>
            <button data-action="add-2">
                 <svg class="kg-icon-button-svg">
                     <use href="/images/svg/dpicons.svg#add"></use>
                 </svg>
            </button>
        </li>
    </ul>
</kg-action-button>
```

Скрипт

```
let actionButton = document.getElementById("actionButton") as DPElementActionButton;
if (actionButton) {
    actionButton.addEventListener("changed", (e) => {
        let target = e.target as DPElementActionButton;
         if (target) {
             console.log(target.selectedAction);
         }
    actionButton.addEventListener("dpadd1", (e) => {
        let target = e.target as DPElementActionButton;
         if (target) {
             console.log(target.selectedAction);
         }
    actionButton.addEventListener("dpadd2", (e) => {
        let target = e.target as DPElementActionButton;
         if (target) {
             console.log(target.selectedAction);
         }

     })
}
```

## Выпадающий список с кнопками - kg-dropdown

*Внешний вид*

Выпадающий список с кнопками

```
<div class="kg-form-row">
    <kg-dropdown-button>
        <button class="kg-icon-button" title="Удалить 1" data-action="remove1">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
```

```
        </button>
        <button class="kg-icon-button" title="Удалить 2" data-action="remove2">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
        <button class="kg-icon-button" title="Удалить 3" data-action="remove3">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
    </kg-dropdown-button>
</div>
```

*Интерфейс / Свойства / Методы*

```
interface DPElementDropdownButton extends HTMLElement {
   /** значение блокировки компонента(true - заблокированы, false - активный). По
умолчанию - false */
   disabled: boolean;
   /** получить значение id атрибута нажатой кнопки. */
   selectedId(): string;
   /** получить значение data-action атрибута нажатой кнопки. */
   selectedAction(): string;
   /** блокировка/включение дочерней кнопки компонента
   * @param action - значение атрибута data-action кнопки, которую нужно
заблокировать/включить
   * @param status - true - для блокировки, false - для включения
   */
   Disable(action: string, status: boolean): Function;
}
```

Пример использования с преобразованием типа

```
const element = document.querySelector("kg-dropdown-button") as
DPElementDropdownButton;
element.disabled = "true";
const disabled = element.disabled;
```

*Свойства*

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **disabled** | boolean Заблокировать элемент(главную кнопку) | Да | Да | Да | false |
| **selectedAction** | string значение data-action(action в случае главной кнопки) атрибута нажатой кнопки | Да | Нет | Нет | |
| **selectedId** | string значение id атрибута нажатой кнопки | Да | Нет | Нет | |

*Методы*

| Название | Описание | Параметры | Возврат | Async |
|---|---|---|---|---|
| **Disable** | Блокировка/включение дочерней кнопки компонента | **action** - Значение атрибута data-action кнопки, которую нужно заблокировать/включить | Нет | Нет |

| | | | | |
|---|---|---|---|---|
| | | **status** - true - для блокировки, false для включения | | |

*Динамическое добавление: Пример*

Для использования в TypeScript требуется преобразование типа ключевым словом as ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let element = document.createElement("kg-action-button") as DPElementDropdownButton;
element.disabled = "true";
```

*События*

У компонента есть основное событие "**changed**", которое вызывается при клике на дочерние кнопки.

Также при клике на дочернюю кнопку и наличии у нее дата-атрибута data-action вызывается событие, которое формируется из приставки "dp" + значения дата-атрибута data-action. Например, "dp" + "remove1" = "**dpremove1**".

Разметка

```
<kg-dropdown-button id="dropdownButton">
        <button class="kg-icon-button" title="Удалить 1" data-action="remove1">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
        <button class="kg-icon-button" title="Удалить 2" data-action="remove2">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
        <button class="kg-icon-button" title="Удалить 3" data-action="remove3">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
    </kg-dropdown-button>
```

Скрипт

```
let dropdownButton = document.getElementById("dropdownButton") as
DPElementDropdownButton;
if (dropdownButton) {
    dropdownButton.addEventListener("changed", (e) => {
        let target = e.target as DPElementDropdownButton;
         if (target) {
             console.log(target.selectedAction);
         }
    dropdownButton.addEventListener("dpremove1", (e) => {
        let target = e.target as DPElementDropdownButton;
         if (target) {
             console.log(target.selectedAction);
```

```
        }
    dropdownButton.addEventListener("changed", (e) => {
        let target = e.target as DPElementDropdownButton;
         if (target) {
             console.log(target.selectedAction);
         }
    dropdownButton.addEventListener("changed", (e) => {
        let target = e.target as DPElementDropdownButton;
         if (target) {
             console.log(target.selectedAction);
         }

     })
}
```

## Панель кнопок - kg-toolbar

### Внешний вид

*Панель кнопок*

*Панель кнопок, ширина которой меньше суммы ширин детей панели*

```
<div class="kg-form-row">
    <kg-toolbar>
        <button class="kg-icon-button" title="Удалить 1" data-action="remove-1">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
        <button class="kg-icon-button" title="Удалить 2" data-action="remove-2">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
        <kg-checkbox-button icon="add" title="Добавить"></kg-checkbox-button>
    </kg-toolbar>
</div>
<div class="kg-form-row">
    <kg-toolbar style="width: 100px;">
        <button class="kg-icon-button" title="Удалить 1" data-action="remove-1">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
        <button class="kg-icon-button" title="Удалить 2" data-action="remove-2">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
```

```
        <kg-checkbox-button icon="add" title="Добавить"></kg-checkbox-button>
    </kg-toolbar>
</div>
```

*Динамическое наполнение панели кнопок*

Для динамического наполнения панели элементами нужно передать в сеттер **items** массив объектов с информацией об элементе.

Для все элементов объект обязательно должен содержать свойство **type**, в котором хранится строка с тэгом HTML элемента. Например, **"kg-radio-button"** или **"button"**.

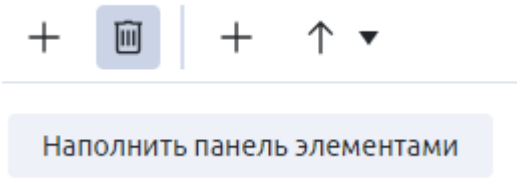
Для элемента **button** объект должен содержать свойства: icon - id иконки в спрайте, action и/или id, рекомендуется использовать action. Опционально: title - текст отображаемый при наведении.

Для элемента **kg-radio-button** объект должен содержать свойства: icon - id иконки в спрайте, action и/или id, рекомендуется использовать action. Опционально: title - текст отображаемый при наведении, value - значение компонента, iconReplace - подменная иконка компонента(появляется при value равном true).

Для элемента **kg-radio-button** объект должен содержать свойства: icon - id иконки в спрайте, action и/или id, рекомендуется использовать action. Опционально: title - текст отображаемый при наведении, value - значение компонента, name - значение атрибута name внутреннего элемента input type="radio".

Для элемента **kg-dropdown-button** объект должен содержать свойства: buttons - массив объектов с информацией об элементе. Для объектов массива buttons правила те же, что и для объектов массива items. Опционально: id - id элемента.

Для элемента разделителя объект должен содержать только свойство type со значение **"separator"**.




```
<div class="kg-form-row" id="container"></div>
    <button id="toolbarButton" class="kg-button-simple">Наполнить панель
элементами</button>
</div>
```

```
const container = document.getElementById("container");
const toolbarButton = document.getElementById("toolbarButton");
const toolbarDynamic = document.createElement("kg-toolbar") as DPElementToolbar;
const ARRAY = [
     { type: "button", icon: "add", title: "button", action: "close" },
     { type: "kg-checkbox-button", icon: "remove", value: true, title: "check1",
action: "remove"},
     { type: "separator" },
     { type: "kg-radio-button", icon: "add", name: "add", title: "radio1", action:
"add"},
     { type: "kg-dropdown-button",
     buttons: [
         { type: "button", icon: "up-big", title: "button1", action: "up" },
```

```
        { type: "button", icon: "down-big", title: "button2", action: "down" },
    ]
    },
],
container.append(toolbarDynamic);
toolbarButton.addEventListener("click", () => toolbarDynamic.items = ARRAY);
```

*Интерфейс / Свойства / Методы*

```
interface DPElementToolbar extends HTMLElement {
   /** значение блокировки дочерних элементов компонента(true - заблокированы, false
- активный). По умолчанию - false. */
   disabled: boolean;
   /** значение id атрибута нажатой кнопки. */
   selectedId: string;
   /** значение data-action(action в случае кастомных кнопок) атрибута нажатой
кнопки. */
   selectedAction: string;
   /** массив, по которому были динамически созданы дочерние элементы тулбара*/
   items: Array<any>;
   /** блокировка/включение элемента тулбара
   * @param action - значение геттера action/data-action атрибута элемента, которую
нужно заблокировать/включить
   * @param status - true - для блокировки, false - для включения
   */
   Disable(action: string, status: boolean): Function;
   /**
   * добавление элемента в тулбар
   * @param value - объект с информацией о кнопке
   */
   Add(value: any): Function
   /**
   * удаление дочернего элемента компонента
   * @param action - значение геттера action/data-action атрибута элемента, который
нужно удалить
   */
   Remove(action: any): Function;
}
```

Пример использования с преобразованием типа

```
const element = document.querySelector("kg-toolbar") as DPElementToolbar;
element.disabled = "true";
const disabled = element.disabled;
```

Свойства

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **disabled** | boolean Значение блокировки дочерних элементов компонента(true - заблокированы, false - активный). По умолчанию - false. | Да | Да | Да | false |

| | | | | | |
|---|---|---|---|---|---|
| **items** | Array массив, по которому были динамически созданы дочерние элементы тулбара | Да | Да | Нет | |
| **selectedAction** | string значение data-action(action в случае кастомных кнопок) атрибута нажатой кнопки | Да | Нет | Нет | |
| **selectedId** | string значение id атрибута нажатой кнопки | Да | Нет | Нет | |

### Методы

| Название | Описание | Параметры | Возврат | Async |
|---|---|---|---|---|
| **Add** | добавление кнопки в тулбар | **value** - Объект с информацией о добавляемой кнопке. Правила для объекта те же, что и для и объектов массива items. | Нет | Нет |
| **Add** | удаление дочернего элемента компонента | **action** - значение геттера action/data-action атрибута элемента, который нужно удалить | Нет | Нет |
| **Disable** | Блокировка/включение дочерней кнопки компонента | **action** - Значение атрибута data-action/action кнопки, которую нужно заблокировать/включить **status** - true - для блокировки, false для включения | Нет | Нет |

### *Динамическое добавление: Пример*

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let element = document.createElement("kg-action-button") as DPElementToolbar;
element.disabled = "true";
```

### *Событие*

### Разметка

```
<kg-toolbar id="toolbar">
        <button class="kg-icon-button" title="Удалить 1" data-action="remove-1">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
        <button class="kg-icon-button" title="Удалить 2" data-action="remove-2">
            <svg class="kg-icon-button-svg">
                <use href="/images/svg/dpicons.svg#remove"></use>
            </svg>
        </button>
        <kg-checkbox-button icon="add" title="Добавить"></kg-checkbox-button>
    </kg-toolbar>
```

### Скрипт

```
let toolbar = document.getElementById("toolbar") as DPElementToolbar;
```

```
if (toolbar) {
    toolbar.addEventListener("changed", (e) => {
        let target = e.target as DPElementDropdownButton;
         if (target === toolbar) {
             console.log(target.selectedAction);
         }

     })
}
```

## Поле ввода

### Поле input

*Простое поле*

*Поле с подсказкой*

Введите текст...

```
<div class="kg-form-row">
    <label class="kg-form-row-label">Простое поле</label>
    <input type="text" class="kg-input"/>
</div>
<div class="kg-form-row">
    <label class="kg-form-row-label">Поле с подсказкой</label>
    <input type="text" class="kg-input" placeholder="Введите текст..." />
</div>
```

### Поле textarea

*Поле текста*

Текст

*Поле с подсказкой*

Введите текст...

```
<div class="kg-form-row">
    <label class="kg-form-row-label">Поле текста</label>
    <textarea class="kg-input-text ">Текст</textarea>
</div>
<div class="kg-form-row">
    <label class="kg-form-row-label">Поле с подсказкой</label>
    <textarea class="kg-input-text " placeholder="Введите текст..."></textarea>
</div>
```

## Поле со списком - kg-select

### Внешний вид

*Выпадающий список обычный*

*Выпадающий список сгруппированный*

*Выпадающий список древовидный*

*Выпадающий список с мультивыбором*

### Разметка

```
<div class="kg-form-row">
    <label class="kg-form-row-label">Выпадающий список обычный</label>
    <kg-select id="select" dataTextField="name" dataValueField="Id"></kg-select>
</div>
<div class="kg-form-row">
    <label class="kg-form-row-label">Выпадающий список сгрупированный</label>
    <kg-select id="selectGroup" group="true" dataTextField="name"
dataValueField="Id"></kg-select>
</div>
<div class="kg-form-row">
    <label class="kg-form-row-label">Выпадающий список древовидный</label>
    <kg-select id="selectTree" tree="true" dataTextField="name"
dataValueField="Id"></kg-select>
</div>
<div class="kg-form-row">
    <label class="kg-form-row-label">ыпадающий список с мультивыбором</label>
    <kg-select id="selectMulti" multi="true" dataTextField="name"
dataValueField="Id"></kg-select>
</div>
```

База данных

```
let data = [
    { name: "Атрибуты", Id: "33d1", ParentId: null },
    { name: "Модель", Id: "11sa", ParentId: null },
    { name: "Список", Id: "zxc0", ParentId: null },
    { name: "Объект", Id: "obj12", ParentId: null },
    { name: "Атрибут 01", Id: "11asd", ParentId: "33d1" },
    { name: "Атрибут 02", Id: "002aa1", ParentId: "33d1" },
    { name: "Модель 01", Id: "002aa13we", ParentId: "11sa" },
    { name: "Модель 02", Id: "002aa1112", ParentId: "11sa" },
    { name: "Список 01", Id: "zxc012", ParentId: "zxc0" },
    { name: "Список 02", Id: "zxc0aa", ParentId: "zxc0" },
    { name: "Объект 01", Id: "obj12xxx", ParentId: "obj12" },
]
```

Скрипт

```
let select = document.getElementById("select") as DPElementSelect
if (select) {
   select.dataSource = new DPDataSource({
       list: data,
       schema: {
           model: {
               id: "Id",
               parentId: "ParentId"
           }
       }
   })
}
let selectGroup = document.getElementById("selectGroup") as DPElementSelect
if (selectGroup) {
   selectGroup.dataSource = new DPDataSource({
       list: dataGroup,
       schema: {
           model: {
               id: "Id",
               parentId: "ParentId"
           }
       }
   })
}
let selectTree = document.getElementById("selectTree") as DPElementSelect
if (selectTree) {
   selectTree.dataSource = new DPDataSource({
       list: dataGroup,
       schema: {
           model: {
               id: "Id",
           }
       }
   })
}
```

```
let selectMulti = document.getElementById("selectMulti") as DPElementSelect
if (selectMulti) {
   selectMulti.dataSource = new DPDataSource({
       list: data,
       schema: {
           model: {
               id: "Id",
           }
       }
   })
}
```

Интерфейс / Свойства

```
interface DPElementSelect extends HTMLElement {
    /** Передать/получить объект класса (содержащий исходные данные) для отображения
списка */
    dataSource: DPDataSource;
    /** Установить/получить значение, передаем string если работаем со списком без
мультивыбора, если с мультивыбором то передаем массив из строк значений */
    value: string | Array<any>;
    /** Установить/получить состояние блокировки календаря, true - заблокировать,
false - разблокировать, по умолчанию false */
    disabled: boolean;
    /** Установить/получить индекс выбранного элемента */
    select: number;
    /** Установить/получить текст выбранного элемента */
    text: string;
    /** Запрещает редактирование поля, возвращает булевое значение, по умолчанию -
false */
    readonly: boolean;
    /** Установить/получить наименование ключа объекта модели, в котором хранится
значение.  */
    dataValueField: string;
    /** Установить/получить наименование ключа объекта модели, в котором хранится
текст. */
    dataTextField: string;
    /** Включить мультивыбор, по умолчанию - false */
    multi: boolean;
    /**  Включить древовидный вид, по умолчанию - false */
    tree: boolean;
    /**  Включить группировку, по умолчанию - false */
    group: boolean;
    /**  Включить кнопку очистки поля ввода, по умолчанию - false */
    clearButton: boolean;
    /**  Получить id родителя выбранного элемента */
    parent: string | number;
    /** Получить модель выбранного элемента */
    selected: any;
    /** Возвращает булевое значение, true - открыто, false - закрыто, по умолчанию -
false */
    isOpen: boolean;
```

```
    /** Метод для открытия выпадающего списка */
    Open(): Function;

    /** Метод для закрытия выпадающего списка */
    Close(): Function;
}
```

## Описание

### *Параметры инициализации*

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **clearButton** | boolean | Включить кнопку очистки поля ввода | false |
| **dataTextField** | string | Установить имя ключа объекта модели, в котором хранится текст | |
| **dataValueField** | string | Установить имя ключа объекта модели, в котором хранится значение | |
| **group** | boolean | Включить группировку | false |
| **multi** | boolean | Включить мультивыбор | false |
| **tree** | boolean | Включить древовидный вид | false |

## Свойства

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **dataSource** | DPDataSource - Передать/получить объект класса (содержащий исходные данные) для отображения списка | Да | Да | Да | null |
| **disabled** | boolean - Заблокировать элемент | Да | Да | Да | false |
| **parent** | string \| number - Получить id родителя выбранного элемента | Да | Нет | Да | |
| **readonly** | boolaen - Запретить редактирования поля | Да | Да | Да | false |
| **select** | number - Установить/получить индекс выбранного элемента | Да | Да | Да | -1 |
| **selected** | any - Получить модель выбранного элемента | Да | Нет | Нет | undefined |
| **text** | string - Установить/получить текст выбранного элемента | Да | Да | Да | |
| **value** | string \| Array{any} - Установить/получить значение, можно передать массив строковых значений | Да | Да | Да | |

## Методы

| Название | Описание | Параметры | Возврат |
|---|---|---|---|
| **Close** | Закрыть выпадающий список | | |
| **Open** | Открыть выпадающий список | | |

| | | | |
|---|---|---|---|
| **dataSource.Add** | Добавить элемент в список | модель элемента | |
| **dataSource.Remove** | Удалить элемент из списка | идентификатор элемента | |

*Динамическое добавление: Пример*

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let select = document.createElement("kg-select") as DPElementSelect;
select.dataSource = new DPDataSource({
    list: this.dataGroup,
    schema: {
        model: {
            id: "Id",
            parentId: "ParentId"
        }
    }
})
select.dataTextField = "name";
select.dataValueField = "Id";
select.clearButton = true;
select.tree = true;
select.multi = true;
select.value = "002aa13we";

let selectBlock = document.getElementById("selectBlock");
if (selectBlock) {
   selectBlock.append(select);
}
```

## События

*Разметка*

```
<kg-select id="selectEvent" dataTextField="name" dataValueField="Id"></kg-select>
```

*Скрипт*

```
/**Находим элемент в DOM */
let select = document.getElementById("selectEvent") as DPElementSelect;
if (select) {
    // Подписываемся на событие
    select.addEventListener("changed", function (e) { console.log(e.target.selected,
"Можем получить модель выбранного элемента") });
}
```

## Поле ввода число - kg-number

### Внешний вид

*Поле ввода число*

```
<div class="kg-form-row">
     <kg-number value="5"></kg-number>
</div>
```

Интерфейс / Свойства

```
interface DPElementsNumber extends HTMLElement {
   /** значение. При получении всегда приходит число. При записи можно передать и
строку, и число. */
   value: number | string;
   /** минимальное числовое значение. */
   min: number;
   /** максимальное числовое значение. */
   max: number;
   /** шаг, на который изменяется числовое значение. */
   step: number;
   /** количество знаков после запятой числового значения компонента. По умолчанию -
3 */
   restrictDecimals: number;
   /** число, на которое умножается числовое значения компонента перед отображением.
По умолчанию - 1 */
   factor: number;
   /** значение блокировки компонента(true - заблокированы, false - активный). По
умолчанию - false */
   disabled: boolean;
   /** значение, округляется ли значение компонента. По умолчанию - false */
   round: boolean;
   /** значение, выделяется ли значение компонента при фокусе. По умолчанию - false
*/
   selectOnFocus: boolean;
   /** значение, отображаются ли стрелки увеличения/уменьшения значения компонента.
По умолчанию - true */
   showArrows: boolean;
   /** значение, можно ли редактировать значение компонента стрелками или вводом. По
умолчанию - false */
   readonly: boolean;
   /** значение, округляется ли значение компонента. По умолчанию - false */
   text: string;
   /** текст внутри текстового поля, который исчезает при получении фокуса. */
   placeholder: string;
   /** текст, отображаемый при наведении на стрелку вверх. */
   upArrowText: string;
   /** текст, отображаемый при наведении на стрелку вниз. */
   downArrowText: string;
   /** Получить формат отображения компонента.
   * По умолчанию “number” – рядом со значением не отображается ничего,
   * “percent” – справа от значения отображается значок “%” ,
   * “currency” – слева от значения отображается значок “₽”
   */
   format(): string;
}
```

*Пример использования с преобразованием типа*

```
const element = document.querySelector("kg-number") as DPElementsNumber;
```

```
element.value = 5;
const value = element.value;
```

## Описание

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **action** | string Действие элемента | Да | Да | Да | |
| **disabled** | boolean Заблокировать элемент | Да | Да | Да | false |
| **downArrowText** | string текст, отображаемый при наведении на стрелку вниз. | Да | Да | Да | |
| **factor** | number Число, на которое умножается числовое значения компонента перед отображением | Да | Да | Да | 1 |
| **format** | string формат отображения компонента. По умолчанию “number” – рядом со значением не отображается ничего, “percent” – справа от значения отображается значок “%”, “currency” – слева от значения отображается значок “₽”. | Да | Да | Да | number |
| **max** | number Максимальное числовое значение | Да | Да | Да | Infinity |
| **min** | number Минимальное числовое значение | Да | Да | Да | -Infinity |
| **placeholder** | string текст внутри текстового поля, который исчезает при получении фокуса. | Да | Да | Да | |
| **readonly** | boolean значение, можно ли редактировать значение компонента стрелками или вводом. | Да | Да | Да | false |
| **restrictDecimals** | number Количество знаков после запятой числового значения | Да | Да | Да | 3 |
| **round** | boolean значение, округляется ли значение компонента | Да | Да | Да | false |
| **selectOnFocus** | boolean значение, выделяется ли значение компонента при фокусе. | Да | Да | Да | false |
| **showArrows** | boolean значение, отображаются ли стрелки увеличения/уменьшения значения компонента. | Да | Да | Да | true |
| **step** | number Шаг, на который изменяется числовое значение | Да | Да | Да | 1 |
| **text** | string текст, отображаемый при наведении на компонент. | Да | Да | Да | |
| **title** | string Подсказка элемента | Да | Да | Да | |
| **upArrowText** | string текст, отображаемый при наведении на стрелку вверх. | Да | Да | Да | |

| | | | | | |
|---|---|---|---|---|---|
| **value** | number \| string Значение. При получении всегда приходит число. При записи можно передать и строку, и число. | Да | Да | Да | 0 |

*Динамическое добавление: Пример*

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
 let element = document.createElement("kg-number") as DPElementsNumber;
 element.value = 5;
```

### Событие

*Разметка*

5

```
<kg-number
 id="number"
 value="5"
></kg-number>
```

### Скрипт

```
let number = document.getElementById("number") as DPElementsNumber;
if (number) {
    number.addEventListener("changed", (e) => {
        let target = e.target as DPElementsNumber;
         if (target) {
             console.log(target.value);
         }
     })
}
```

## Поле поиска - kg-search

### Внешний вид

### Простое поле

Поиск...

### Поле с кнопками группировки

Поиск...

### Поле с кнопками группировки и сортировки

Поиск...

```
<div class="kg-form-row">
    <label class="kg-form-row-label">Простое поле</label>
    <kg-search ></kg-search>
</div>
<div class="kg-form-row">
    <label class="kg-form-row-label">Поле с кнопками групировки</label>
    <kg-search tree="true"></kg-search>
</div>
<div class="kg-form-row">
    <label class="kg-form-row-label">Поле с кнопками групировки и сортировки</label>
```

```
    <kg-search tree="true" sorting="true"></kg-search>
</div>
```

### Интерфейс / Свойства

```
interface DPElementSearch extends HTMLElement {
    /** Включение кнопок сортировки, true - включить, false - выключить */
    sorting: string;
    /**Значение в поле поиска */
    value: string;
    /**Заблокировать элемент. true - заблокировать, false - снять блокировку */
    disabled: boolean;
    /** Включение кнопок свернуть или развернуть, только для чтения */
    tree: string;
}
```

*Пример использования с преобразованием типа*

```
let element = document.querySelector("kg-search") as DPElementSearch;

element.value = "текст";
let value = element.value;

element.disabled = true;
let disabled = element.disabled;

let sorting = element.sorting;

let tree = element.tree;
```

### Описание

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **disabled** | Заблокировать элемент | Да | Да | Да | false |
| **sorting** | Включение кнопок сортировки, true - включить, false - выключить | Да | Нет | Да | false |
| **tree** | Включение кнопок свернуть/развернуть, true - включить, false - выключить | Да | Нет | Да | false |
| **value** | Значение в поле поиска | Да | Да | Нет | |

*Динамическое добавление: Пример*

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let element = document.createElement("kg-search") as DPElementSearch;
element.tree = true
```

### События

*Разметка*




```
<kg-search id="searchEvents" tree="true" sorting="true"></kg-search>
```

### Скрипт

```
/**Находим элемент в DOM */
```

```
let search = document.getElementById("searchEvents") as DPElementSearch;
if (search) {
    //Нажатие кнопки Свернуть все
    search.addEventListener("collapse", function (e) { console.log(e) });
    // Нажатие кнопки Развернуть все
    search.addEventListener("expand", function (e) { console.log(e) });
    //Нажатие кнопки Поиск
    search.addEventListener("search", function (e) { console.log(e) });
    //Нажатие кнопки Сортировка по возрастанию
    search.addEventListener("ascending", function (e) { console.log(e) });
    //Нажатие кнопки Сортировка по убыванию
    search.addEventListener("descending", function (e) { console.log(e) });
    //Срабатывает при введение или изменение текста в поле поиска
    search.addEventListener("input", function (e) { console.log(e) });
}
```

## Выбор даты - kg-datepicker

### Внешний вид

*Календарь без функции выбора времени*

25.10.2023

*Календарь с функцией выбора времени*

22.10.2022 00:00

```
<div class="kg-form-row">
    <label class="kg-form-row-label">Календарь без функции выбора времени</label>
    <kg-datepicker format="dd.MM.yyyy" time="false"></kg-datepicker>
</div>
<div class="kg-form-row">
    <label class="kg-form-row-label">Календарь с функцией выбора времени</label>
    <kg-datepicker format="dd.MM.yyyy HH:mm" value="22.10.2022"></kg-datepicker>
</div>
```

### Интерфейс / Свойства

```
interface DPElementDatePicker extends HTMLElement {
    /** Устанавливает/возвращает значение календарю, по умолчанию new Date() */
    value: Date | string;
    /** Включает/отключает время, возвращает булевое значение, по умолчанию - true
*/
    time: boolean;
    /** Устанавливает/возвращает состояние блокировки календаря, true -
заблокировать, false - разблокировать, по умолчанию false */
    disabled: boolean;
    /** Устанавливает/возвращает формат даты, по умолчанию - "DD.MM.YYYY HH:mm" */
    dateFormat: string;
    /** Устанавливает/возвращает формат времени во вкладке выбора времени, по
умолчанию - "HH:mm" */
    timeFormat: string;
    /** Запрещает редактирование поля, возвращает булевое значение, по умолчанию -
false */
    readonly: boolean;
```

```
    /** Устанавливает/возвращает максимально допустимую дату для выбора, по
умолчанию - new Date(2099, 11, 31) */
    max: boolean;
    /** Устанавливает/возвращает минимально допустимую дату для выбора, по умолчанию
- new Date(1900, 0, 1) */
    min: boolean;
    /** Устанавливает максимально допустимое время до текущего, по умолчанию - "",
возвращает выбранное время */
    maxTime: boolean;
    /** Включает/отключает анимацию календаря, по умолчанию - true */
    animation: boolean;
}
```

*Пример использования с преобразованием типа*

```
let element = document.querySelector("kg-datepicker") as DPElementDatePicker;

element.value = new Date();
let value = element.value;

element.disabled = true;
let disabled = element.disabled;

element.time = false;
let time = element.time;

element.dateFormat = "dd.MM.yyyy HH:mm:ss";
let dateFormat = element.dateFormat;

element.timeFormat = "HH:mm:ss";
let timeFormat = element.timeFormat;

element.readonly = true;
let readonly = element.readonly;

element.maxTime = true;
let maxTime = element.maxTime;

element.max = new Date();
let max = element.max;

element.min = new Date();
let min = element.min;
```

## Описание

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **dateFormat** | string - Установить формат отображения даты и времени | Да | Да | Нет | DD.MM.YYYY HH:mm |
| **disabled** | boolean - Заблокировать элемент | Да | Да | Да | false |
| **format** | string - Установить формат отображения даты и времени | Да | Нет | Да | DD.MM.YYYY HH:mm |

| | | | | | |
|---|---|---|---|---|---|
| **interval** | number - Установить интервал отображения времени | Да | Да | Да | 30 |
| **max** | Date - Установить максимально допустимую дату | Да | Да | Да | new Date(2099, 11, 31) |
| **maxTime** | boolean - Ограничить время до текущего | Да | Да | Да | |
| **min** | Date - Установить минимально допустимую дату | Да | Да | Да | new Date(1900, 0, 1) |
| **readonly** | boolaen - Запретить редактирования поля | Да | Да | Да | false |
| **time** | boolean false - Отключить выбор времени | Да | Да | Да | true |
| **timeFormat** | string Установить формат отображения времени во вкладке выбора времени | Да | Да | Да | HH:mm |
| **value** | Date - Установить значение в виде даты | Да | Да | Да | new Date() |

*Примечание по свойствам*

Свойство format можно использовать только в виде атрибута, для того что бы изменять формат даты в js, необходимо использовать свойство dateFormat

Свойство maxTime принимает в себя boolean значения, но возвращает максимально допустимое время в строковом формате, данное свойство можно использовать только если max = true

### Динамическое добавление: Пример

Для использования в TypeScript требуется преобразование типа ключевым словом **as** ниже описанным интерфейсом.

```
/// <reference path="../kg.elements.d.ts" />
let element = document.createElement("kg-datepicker") as DPElementDatePicker;

element.dateFormat = "dd.MM.yyyy HH:mm:ss"
element.timeFormat = "dd.MM.yyyy HH:mm:ss"
element.min = new Date(2019, 0, 15)
element.max = new Date()
element.maxTime = true
element.readonly = true
```

### События

*Разметка*




```
<kg-datepicker id="datepicker" format="dd.MM.yyyy HH:mm"></kg-datepicker>
```

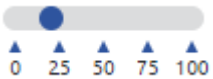
### Скрипт

```
/**Находим элемент в DOM */
let datepicker = document.getElementById("datepicker") as DPElementDatePicker;
if (datepicker) {
    // Выбора даты или времени
    search.addEventListener("changed", function (e) { console.log(e) });
}
```

## Диапазон - kg-range

### Внешний вид

*Диапазон горизонтальный с 1 слайдером и метками в виде стрелок*




*Диапазон вертикальный с 2 слайдерами, метками в виде шкалы и отключенными числовыми значениями*

*Диапазон горизонтальный с 2 слайдерами, с отключенными метками*

```
<div class="kg-form-row">
     <kg-range valueFirst="20"></kg-radio-button>
</div>
<div class="kg-form-row">
     <kg-range valueFirst="20" valueSecond="70" double="true" stepMode="scale"
vertical="true" numbers="false"></kg-range>
</div>
<div class="kg-form-row">
     <kg-range valueFirst="20" valueSecond="70" double="true" marks="false"></kg-
range>
</div>
```

### Интерфейс / Свойства

```
interface DPElementRange extends HTMLElement {
   /** числовое значение слайдеров */
   value: number | string | Array<number | null>;
   /** значение блокировки компонента(true - заблокированы, false - активный). По
умолчанию - false */
   disabled: boolean;
   /** количество слайдеров компонента. */
   double: boolean;
   /** режим отображения компонента. */
   vertical: boolean;
   /** статус отображения меток и интервалов компонента. */
   marks: boolean;
   /** статус отображения отображения чисел возле меток компонента. */
   numbers: boolean;
   /** минимальное числовое значение слайдеров. */
   min: number;
   /** максимальное числовое значение слайдеров. */
   max: number;
   /** количество интервалов компонента. */
```

```
    intervals: number;
    /** режим отображения меток компонента. */
    stepMode: string;
}
```

### Пример использования с преобразованием типа

```
const element = document.querySelector("kg-range") as DPElementRange;

element.value = [20, 70];
const value = element.value;

element.min = 0;
const min = element.min;

element.max = 100;
const max = element.max;

element.intervals = 4;
 const intervals = element.intervals;

element.disabled = false;
const disabled = element.disabled;

element.double = false;
const double = element.double;

element.vertical = false;
const vertical = element.vertical;

element.marks = true;
const marks = element.marks;

element.numbers = true;
const numbers = element.numbers;

element.stepMode = "scale";
const stepMode = element.stepMode;
```

### Описание

| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **disabled** | boolean Заблокировать элемент | Да | Да | Да | false |
| **double** | boolean Колличество слайдеров элемента | Да | Да | Да | false |
| **intervals** | number Количество интервалов элемента | Да | Да | Да | 4 |
| **marks** | boolean Статус отображения меток и интервалов элемента | Да | Да | Да | true |
| **max** | number Максимальное значение слайдеров элемента | Да | Да | Да | 100 |
| **min** | number Минимальное значение слайдеров элемента | Да | Да | Да | 0 |

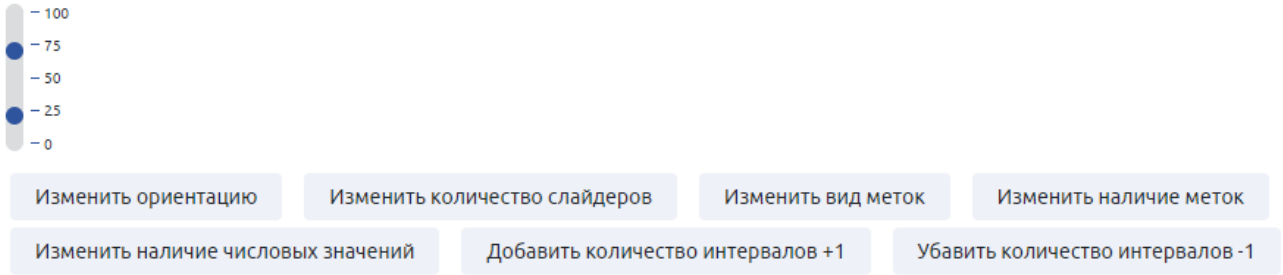
| | | | | | |
|---|---|---|---|---|---|
| **numbers** | boolean Статус отображения чисел возле меток элемента | Да | Да | Да | true |
| **stepMode** | string режим отображения меток компонента - "arrow"/"scale". | Да | Да | Да | arrow |
| **value** | Array \| number \| string Числовое значение слайдеров. при получении приходит массив чисел [valueFirst, valueSecond]. при записи можно передать:число, строку, которую можно перевести в число - значение запишется в valueFirst(первый слайдер), массив чисел(на месте одного из чисел можно передать null) - значения запишутся соответственно value[0] в valueFirst, value[1] в valueSecond. при передаче массива чисел value[0] должно быть меньше value[1] минимум на 1. | Да | Да | нет | [0, 1] |
| **valueFirst** | number Числовое значение 1го слайдера. | нет | нет | Да | 0 |
| **valueSecond** | number Числовое значение 2го слайдера. | нет | нет | Да | 1 |
| **vertical** | boolean Режим отображения элемента | Да | Да | Да | false |

*Динамическое добавление: Пример*

```
/// <reference path="../kg.elements.d.ts" />
 let element = document.createElement("kg-range") as DPElementRange;
 element.value = 20;
```

Событие

*Разметка*




```
<kg-range
 valueFirst="20"
 valueSecond="70"
 double="true"
 stepMode="scale"
 vertical="true"
></kg-radio-button>
```

Скрипт

```
let range = document.getElementById("range") as DPElementRange;
if (range) {
    range.addEventListener("changed", (e) => {
        let target = e.target as DPElementRange;
```

```
        if (target) {
            console.log(`Значение 1го слайдера <b>${target.value[0]}</b>. Значение
2го слайдера <b>${target.value[1]}</b>`);
        }
    })
}
```

## Прогресс бар - kg-progress

### Внешний вид

*Прогресс бар*

20%

```
<div class="kg-form-row">
    <kg-progress value="20"></kg-progress>
</div>
```

### Интерфейс / Свойства

```
interface DPElementProgress extends HTMLElement {
   /** значение компонента.По умолчанию - 0. */
   value: number;
   /** минимальное значение компонента. По умолчанию - 0. */
   min: number;
   /** максимальное значение компонента. По умолчанию - 100. */
   max: number;
   /** значение блокировки компонента. true/false, true - заблокирован, false -
активен. По умолчанию - false. */
   disabled: boolean;
   /** статус наличия анимации компонента. true/false, true - анимирован, false - не
анимирован. По умолчанию - true. */
   animation: boolean;
}
```

*Пример использования с преобразованием типа*

```
const element = document.querySelector("kg-progress") as DPElementProgress;

element.value = 20;
 const value = element.value;

element.min = 0;
const min = element.min;

element.max = 100;
const max = element.max;

element.animation = true;
const animation = element.animation;
```

Описание

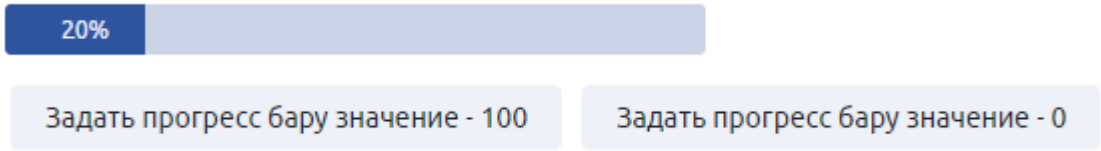
| Свойство | Описание | get | set | Атрибут | По умолчанию |
|---|---|---|---|---|---|
| **animation** | boolean Заблокировать элемент | Да | Да | Да | false |
| **disabled** | boolean Статус наличия анимации элемента | Да | Да | Да | true |
| **max** | number Максимальное значение элемента | Да | Да | Да | 100 |
| **min** | number Минимальное значение элемента | Да | Да | Да | 0 |
| **value** | number Значение элемента | Да | Да | Да | 0 |

*Динамическое добавление: Пример*

```
/// <reference path="../kg.elements.d.ts" />
 let element = document.createElement("kg-progress") as DPElementProgress;
 element.value = 20;
```

Работа с прогресс баром

*Разметка*




```
<kg-progress id="progress" value="20"></kg-progress>
```

Скрипт

```
const progress = document.getElementById("progress") as DPElementProgress;
const progress100Button = document.getElementById("progress100Button");
const progress0Button = document.getElementById("progress0Button");
if (progress) {
    progress100Button.addEventListener("click", () => progress.value = 100);
    progress0Button.addEventListener("click", () => progress.value = 0);
}
```

# Диаграммы

## Начало работы

### Создание с добавлением элемента через html

```
<div class="kg-form-row" style="width: 100%; height: 500px">
   <kg-chart id="chart"></kg-chart>
</div>
```

```
let Chart = document.getElementById("chart")

if (Chart) {
   Chart.dataSource = {
```

```
        series: [
            {
                name: "Ряд данных",
                data: getData(),
                color: "#428B45",
                markerSize: 0
            },
        ]
    }
}
function getData() {
    let limit = 100;
    let dataPoints = [];
    let y: number = 100;
    for (let i = 0; i < limit; i += 1) {
        y += Math.round(Math.random() * 10 - 5);
        dataPoints.push([i, y]);
    }
    return dataPoints;
}
```

Создание с добавлением элемента через скрипт

```
<div class="kg-form-row" style="width: 100%; height: 500px">
</div>
```

```
let container = document.getElementById("chartContainer");
if (container) {
    const chart = document.createElement("kg-chart")

    chart.dataSource = {
        series: [
            {
                name: "Ряд данных",
                data: getData(),
                color: "#ffa615",
                markerSize: 0
            },
        ]
```

```
    }

    container.append(chart)
}

function getData() {
    let limit = 100;
    let dataPoints = [];
    let y: number = 100;
    for (let i = 0; i < limit; i += 1) {
        y += Math.round(Math.random() * 10 - 5);
        dataPoints.push([i, y]);
    }
    return dataPoints;
}
```

## Конфигурация

### Общее

```
chart.options = {
    chart: {
        border: true,
        navigator: {
            enable: true
        }
    },
    line: {
        color: "#ffa615",
        fill: true,
        spline: true,
    }
}
```

*Настройки графика*

Для указания настроек, необходимо в свойство options передать объект с настройками графика

Определяются в: `options.chart`

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **backgroundColor** | string | Цвет заливки подложки диаграммы | #ffffff |

| | | | |
|---|---|---|---|
| **backgroundPlotColor** | string | Цвет заливки области просмотра диаграммы | #ffffff |
| **border** | boolean | Отображение границы диаграммы | false |
| **borderColor** | string | Цвет границы диаграммы | #0C2452 |
| **decimation** | boolean | Прореживание данных | false |
| **inverted** | boolean | Перевернутый график | false |
| **navigator** | object | Навигатор, для перемещения по графику, подробнее | |
| **padding** | number \| object | Внутренние отступы диаграммы, подробнее | 0 |
| **pan** | boolean | Прокрутка графика | false |
| **responsive** | boolean | Адаптивность диаграммы | true |
| **type** | KGChartType | Тип графика | line |
| **zoom** | boolean | Приближение графика по области | false |

### *Типы графика*

- line
- scatter
- radar
- bar
- stackedBar
- rangeBar
- gauge
- solidgauge

### Отступы графика

Отступы можно определять отдельно для каждой стороны: `top`, `left`, `right`, `bottom`

```
Chart.options = {
   chart: {
      padding: {
         top: 10,
         left: 20,
         right: 5,
         bottom: 15
      }
   }
}
```

### *Настройки навигатора*

Навигатор не используется при типе диаграммы лепестковая или индикатор

`options.navigator` - настройки навигатора

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **enable** | boolean | Включить отображение навигатора | false |
| **height** | number | Высота навигатора | 70 |

### *Глобальные настройки*

Глобальные настройки определяют шрифты и цвета всех элементов на графике

`Chart.global` - глобальные настройки

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **annotationColor** | string | Цвет реперных линий и реперных точек | #CBD4E7 |
| **color** | string | Цвет текстовых элементов на графике | #0C2452 |

| | | | |
|---|---|---|---|
| **fillColor** | string | Цвет заливки элементов, такие как графики и маркеры | #CBD4E7 |
| **font** | object | Настройки шрифта | |
| **selectedPointColor** | string | Цвет выделенных точек | #ffa615 |
| **strokeColor** | string | Цвет линий на графике, в том числе осей и сетки | #CBD4E7 |

`Chart.global.font` - глобальные настройки шрифта

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **family** | string | Семейство шрифта | 'Ubuntu, Helvetica Neue, Helvetica, Arial, sans-serif' |
| **size** | number | Размер шрифта | 14 |
| **style** | string | Стиль шрифта | 'normal' |
| **weight** | string | Начертание шрифта | 'normal' |

### *События*

`loaded` - срабатывает, когда компонент отрисован в DOM

```
Chart.addEventListener("loaded", () => {
    console.log("Компонент отрисован в DOM");
})
```

### Работа с данными

Для передачи данных компоненту используется свойство dataSource

В dataSource можно передвать как экземпляр класса KGChartDataSource так и просто объект, с необходимыми полями

dataSource содержит в себе поля series, labels, schema

В поле series передается массив рядов данных, каждый из которых является объектом со своей собственной конфигурацией и набором данных

В поле labels передаются метки, которые инициализируются по оси X, если это свойство используется, то количество меток должно соответствовать количеству значений в data

```
Chart.dataSource = {
   labels: ["one", "two"]
   series: [{
       data: [10, 20]
   }]
}
```

```
Chart.dataSource = {
   series: [{
       data: [[10, 15], [20, 25]]
   }]
}
```

```
Chart.dataSource = {
   series: [{
       data: [{ x: 10, y: 15 }, { x: 20, y: 25 }]
   }]
}
```

```
Chart.dataSource = new KGChartDataSource({
   series: [{
       data: [{ time: "12.02.2023", value: 250 }, { time: "13.02.2023", value: 120
}]
   }]
   schema: {
       model: {
           x: "time",
           y: "value"
       }
   }
}
```

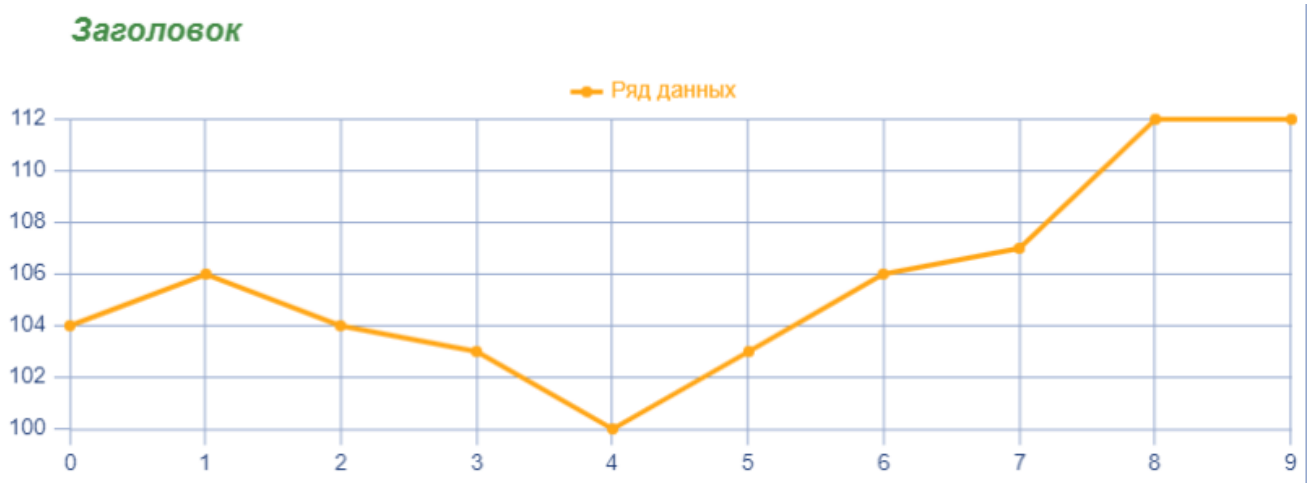
### Конфигурация заголовка

`options.title` - настройка заголовка

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Цвет заголовка | `Chart.global.color` |
| **font** | object | Шрифт заголовка | `Chart.global.font` |
| **horizontalAlign** | 'left' \| 'center' \| 'right' | Положение заголовка по горизонтали | center |
| **padding** | number | Отступы, сверху и снизу | 10 |
| **text** | string | Текст заголовка | '' |
| **verticalAlign** | 'top' \| 'center' \| 'bottom' | Положение заголовка по вертикали | top |

*Пример использования*

```
Chart.options = {
   title: {
       text: "Заголовок",
       color: "#428B45",
       font: {
           size: 20,
           style: "italic",
           weight: "bold"
       },
       horizontalAlign: "left"
   }
}
```




### Конфигурация легенды

`options.legend` - настройка легенды

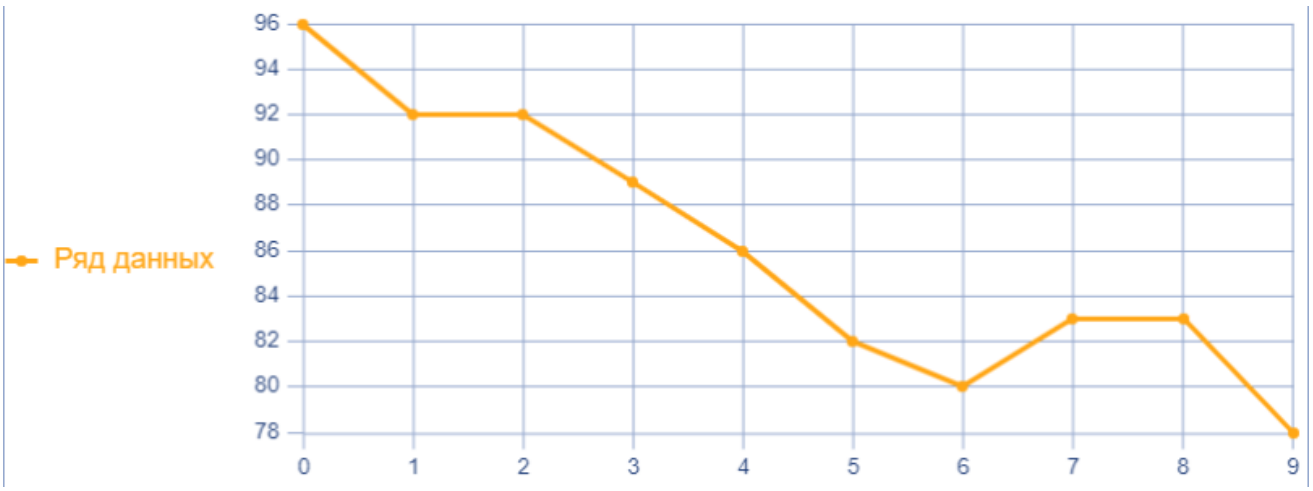
| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **horizontalAlign** | 'left' \| 'center' \| 'right' | Положение легенды по горизонтали | center |
| **label** | object | настройка меток | `Chart.options.legend.label` |
| **padding** | number | Отступы, сверху и снизу | 10 |
| **show** | boolean | Видимость легенды на графике | true |
| **verticalAlign** | 'top' \| 'center' \| 'bottom' | Положение легенды по вертикали | top |

`options.legend.label` - настройка меток легенды

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Цвет заголовка | undefined |
| **font** | object | Шрифт заголовка | `Chart.global.font` |
| **padding** | number | Отступы, сверху и снизу | 10 |

*Пример использования*

```
Chart.options = {
   legend: {
       label: {
           font: {
               size: 20,
           },
           padding: 10
       },
       horizontalAlign: "left",
       verticalAlign: "center"
   }
}
```




## Конфигурация Осей

`options.xAxis` - настройка оси X

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Задает цвет всех элементов оси | undefined |
| **dateFormat** | string | Формат даты | DD.MM.YYYY |
| **grid** | object | Настройки сетки оси | |
| **interval** | number | Интервал значений на оси, если это время, интервал задается в мс | undefined |
| **label** | object | Настройки меток оси | |
| **line** | object | Настройки линии оси | |
| **max** | number \| string \| Date | Максимальное значение на оси | undefined |
| **min** | number \| string \| Date | Минимальное значение на оси | undefined |
| **position** | 'left' \| 'right' \| 'bottom' | Позиция оси, определяет в каком положении будет находится ось | bottom |
| **tick** | object | Настройки отрезков оси | |
| **title** | object | Настройки заголовка оси | |
| **type** | '' \| 'dateTime' \| 'categories' | Тип оси, от корого зависит обработка данных для оси | '' |

`options.xAxis` - настройка оси Y

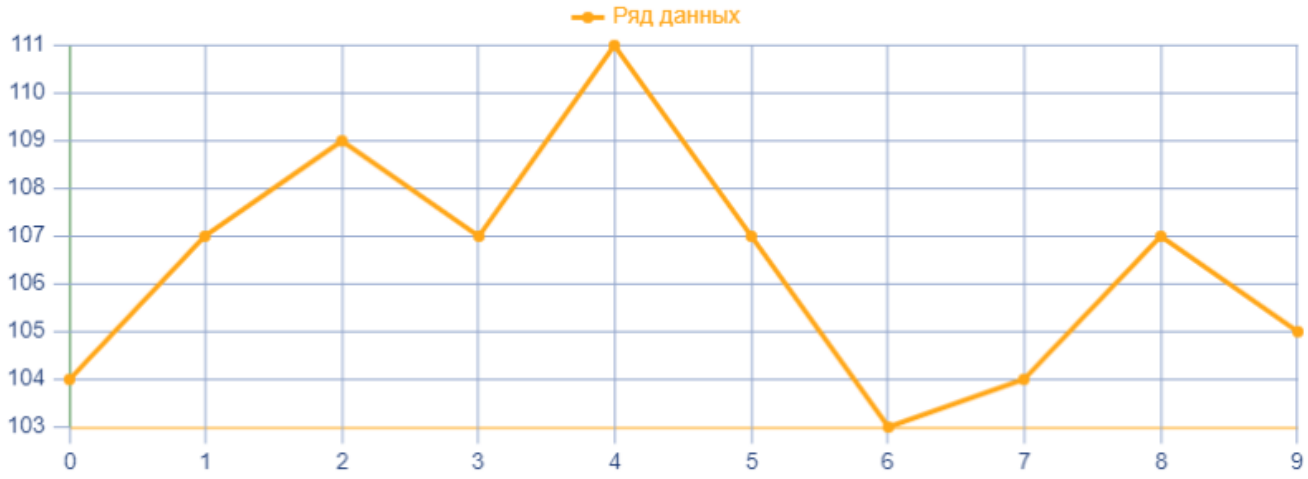
| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Задает цвет всех элементов оси | undefined |
| **grid** | object | Настройки сетки оси | |
| **interval** | number | Интервал значений на оси, если это время, интервал задается в мс | undefined |
| **label** | object | Настройки меток оси | |
| **line** | object | Настройки линии оси | |
| **max** | number \| string \| Date | Максимальное значение на оси | undefined |
| **min** | number \| string \| Date | Минимальное значение на оси | undefined |
| **tick** | object | Настройки отрезков оси | |
| **title** | object | Настройки заголовка оси | |

`options.xAxis.line` - настройка линий оси

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Задает цвет линии оси | undefined |

*Пример использования*

```
Chart.options = {
    xAxis: {
        line: {
           color: "#ffa615"
        }
    },
    yAxis: {
        line: {
            color: "#428B45"
        }
    }
}
```




`options.xAxis.title` - настройка заголовка оси

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Задает цвет заголовка оси | undefined |
| **font** | object | Шрифт заголовка | `Chart.global.font` |
| **padding** | number | Отступы заголовка, сверху и снизу | 5 |
| **text** | string | Текст заголовка оси | '' |

*Пример использования*

```
Chart.options = {
    xAxis: {
        title: {
            text: "Заголовок оси X",
            font: {
                size: 16,
                style: "italic"
            },
            padding: 10,
```

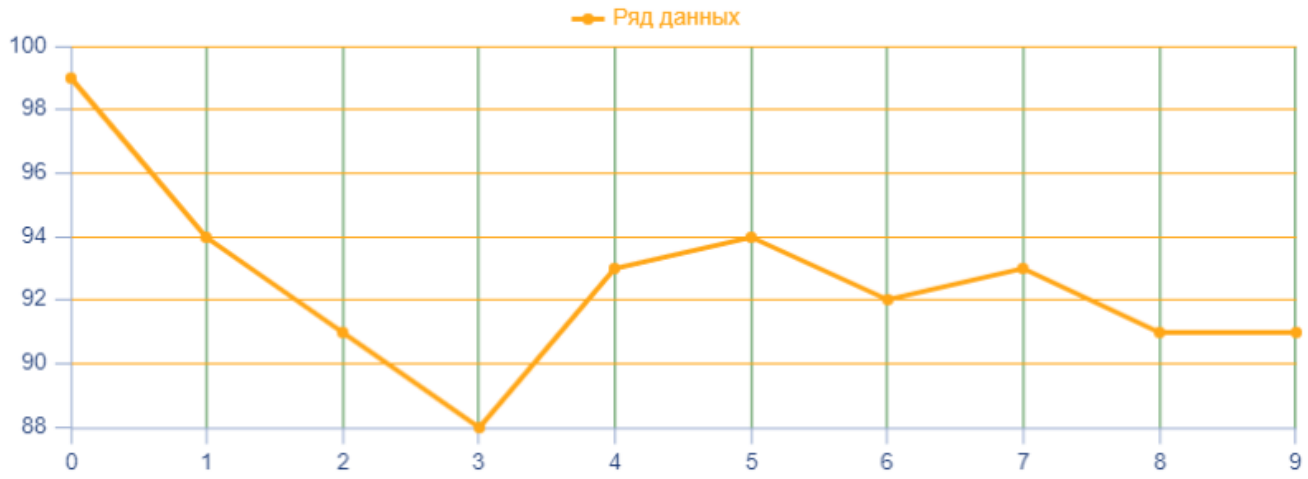
```
            color: "#ffa615"
        }
    },
    yAxis: {
        title: {
            text: "Заголовок оси Y",
            font: {
                size: 12,
                weight: "bold"
            },
            padding: 10,
            color: "#428B45"
        }
    }
}
```

`options.xAxis.grid` - настройка сетки оси

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Задает цвет линии оси | undefined |
| **draw** | boolean | Отрисовать сетку? | true |
| **width** | number | Ширина сетки | 1 |

*Пример использования*

```
Chart.options = {
    xAxis: {
        grid: {
            width: 3,
            color: "#428B45"
        }
    },
    yAxis: {
        grid: {
            color: "#ffa615"
        }
    }
}
```

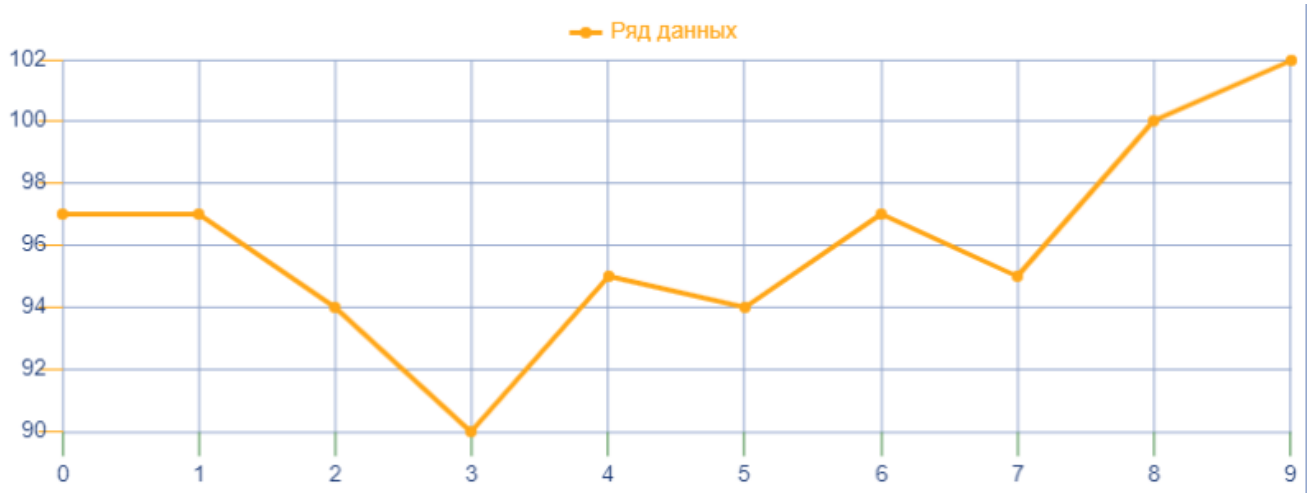



`options.xAxis.tick` - настройка отрезков оси

| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Задает цвет линии оси | undefined |
| **length** | number | Длина отрезка | 10 |
| **max** | number | Максимальное количество отрезков на оси | 10 |
| **width** | number | Ширина отрезка | 1 |

*Пример использования*

```
Chart.options = {
    xAxis: {
        tick: {
            length: 15,
            width: 3,
            color: "#428B45"
        }
    },
    yAxis: {
        tick: {
            length: 5,
            max: 5,
            color: "#ffa615"
        }
    }
}
```




`options.xAxis.label` - настройка меток оси

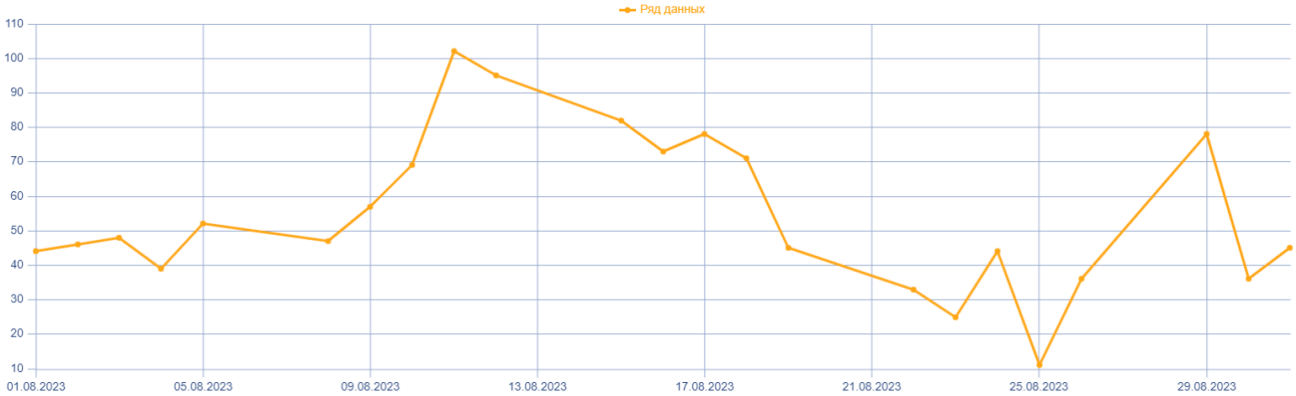
| Название | Тип | Описание | По умолчанию |
|---|---|---|---|
| **color** | string | Задает цвет линии оси | undefined |
| **font** | object | Шрифт метки | `Chart.global.font` |
| **padding** | number | Отступы метки, сверху и снизу | 5 |
| **rotate** | number | Угол наклона метки | 0 |
| **suffix** | string | Приставка к метки | 0 |
| **valueDecimal** | number | Количество знаков после запятой, если это числовое значение | 0 |

*Пример использования*

```
Chart.options = {
    xAxis: {
        label: {
            font: {
                size: 12,
                style: "italic"
            },
            padding: 5,
            rotate: 45,
            valueDecimal: 2,
            suffix: "м",
            color: "#ffa615"
        }
    },
    yAxis: {
        label: {
            font: {
                size: 14,
                style: "italic"
            },
            padding: 10,
            suffix: "км/ч",
            color: "#ffa615"
        }
    }
}
```

*Работа с датами*

Для работы с датами, нужно указать в параметре `options.xAxis.type = "dateTime"`, после этого компонент автоматически начнет рассчитывать данные в датах

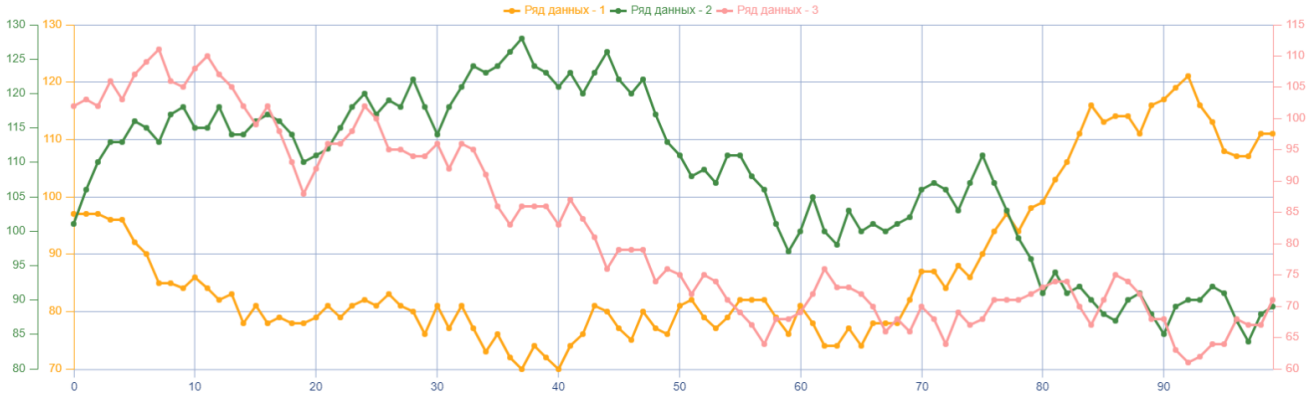



```
Chart.options = {
    chart: {
        border: true,
    },
    xAxis: {
        type: "dateTime"
    }

}

Chart.dataSource = {
    series: [
        {
            name: "Ряд данных",
            data: getData(),
            color: "#ffa615"
        },
    ]
};
function getData() {
```

```
    return [
        { x: new Date(2023, 7, 1), y: 44 },
        { x: new Date(2023, 7, 2), y: 46 },
        { x: new Date(2023, 7, 3), y: 48 },
        { x: new Date(2023, 7, 4), y: 39 },
        { x: new Date(2023, 7, 5), y: 52 },
        { x: new Date(2023, 7, 8), y: 47 },
        { x: new Date(2023, 7, 9), y: 57 },
        { x: new Date(2023, 7, 10), y: 69 },
        { x: new Date(2023, 7, 11), y: 102 },
        { x: new Date(2023, 7, 12), y: 95 },
        { x: new Date(2023, 7, 15), y: 82 },
        { x: new Date(2023, 7, 16), y: 73 },
        { x: new Date(2023, 7, 17), y: 78 },
        { x: new Date(2023, 7, 18), y: 71 },
        { x: new Date(2023, 7, 19), y: 45 },
        { x: new Date(2023, 7, 22), y: 33 },
        { x: new Date(2023, 7, 23), y: 25 },
        { x: new Date(2023, 7, 24), y: 44 },
        { x: new Date(2023, 7, 25), y: 11 },
        { x: new Date(2023, 7, 26), y: 36 },
        { x: new Date(2023, 7, 29), y: 78 },
        { x: new Date(2023, 7, 30), y: 36 },
        { x: new Date(2023, 7, 31), y: 45 }
    ]
}
```

### *Мульти оси*

Компонент поддерживает отображение нескольких осей Y, который соответствует каждому ряду данных




Для отображение больше одной оси Y, необходимо в параметр `options.yAxis` передать массив, который будет содержать модели для определенной оси.

Модель оси будет соотвествовать ряду данных по индексу.

Если осей слишком много или вам нужно их скрыть до взаимодействия пользователя с графиком, вы можете использовать скрытый режим мультишкал, включив параметр `options.chart.hiddenAxisMode`

```
Chart.options = {
```

```
    chart: {
        border: true,
        // hiddenMultiAxisMode: true
    },
    yAxis: [
        {
            color: "#ffa615"
        },
        {
            color: "#428B45",
            grid: {
               draw: false
            }
        },
        {
            color: "#FC9B9B",
            position: "right",
            grid: {
               draw: false
            }
        }
    ]

}

Chart.dataSource = {
    series: [
        {
            name: "Ряд данных - 1",
            data: getData(100),
            color: "#ffa615"
        },
        {
            name: "Ряд данных - 2",
            data: getData(100),
            color: "#428B45"
        },
        {
            name: "Ряд данных - 3",
            data: getData(100),
            color: "#FC9B9B"
        }
    ]
};
function getData(limit) {
   let dataPoints = [];
   let y = 100;

   for (let i = 0; i < limit; i += 1) {
       y += Math.round(Math.random() * 10 - 5);
```

```
        dataPoints.push([i, y])
    }

    return dataPoints;
}
```